

Maria Eduarda Moreira Fernandes

RGM: 42781515

Atividade avaliativa 3

ATIVIDADE PRÁTICA – ANÁLISE DE ALGORITMOS DE ORDENAÇÃO

Etapa 1 e 2

```
import random
import sys

sys.setrecursionlimit(5000)

def bubble_sort(arr):
    v = arr.copy()
    comparacoes = 0
    trocas = 0
    n = len(v)
    for i in range(n):
        for j in range(0, n - i - 1):
            comparacoes += 1
            if v[j] > v[j + 1]:
                v[j], v[j + 1] = v[j + 1], v[j]
                trocas += 1
    return comparacoes, trocas

def insertion_sort(arr):
    v = arr.copy()
    comparacoes = 0
    movimentacoes = 0
    for i in range(1, len(v)):
        chave = v[i]
        j = i - 1
        while j >= 0:
            comparacoes += 1
            if v[j] > chave:
                v[j + 1] = v[j]
                movimentacoes += 1
                j -= 1
            else:
                break
        v[j + 1] = chave
        movimentacoes += 1
    return comparacoes, movimentacoes
```

```
def selection_sort(arr):
    v = arr.copy()
    comparacoes = 0
    trocas = 0
    n = len(v)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            comparacoes += 1
            if v[j] < v[min_idx]:
                min_idx = j
        if min_idx != i:
            v[i], v[min_idx] = v[min_idx], v[i]
            trocas += 1
    return comparacoes, trocas
```

```
def quick_sort(arr):
    v = arr.copy()
    comparacoes = [0]
    movimentacoes = [0]

    def _quick(low, high):
        if low < high:
            pi = _partition(low, high)
            _quick(low, pi - 1)
            _quick(pi + 1, high)

    def _partition(low, high):
        pivo = v[high]
        i = low - 1
        for j in range(low, high):
            comparacoes[0] += 1
            if v[j] <= pivo:
                i += 1
                v[i], v[j] = v[j], v[i]
                movimentacoes[0] += 1
        v[i + 1], v[high] = v[high], v[i + 1]
        movimentacoes[0] += 1
        return i + 1

    _quick(0, len(v) - 1)
    return comparacoes[0], movimentacoes[0]
```

```
tamanhos = [10, 20, 1000]
random.seed(42)
```

```

print("=== ETAPA 3: RESULTADOS COM DADOS ALEATÓRIOS ===")
for n in tamanhos:
    original = [random.randint(1, 10000) for _ in range(n)]
    b_comp, b_trocas = bubble_sort(original)
    i_comp, i_mov = insertion_sort(original)
    s_comp, s_trocas = selection_sort(original)
    q_comp, q_mov = quick_sort(original)

    print(f"\n--- Tamanho: {n} ---")
    print(f"Bubble Sort -> Comparações: {b_comp} | Trocas: {b_trocas}")
    print(f"Insertion Sort -> Comparações: {i_comp} | Movimentações: {i_mov}")
    print(f"Selection Sort -> Comparações: {s_comp} | Trocas: {s_trocas}")
    print(f"Quick Sort -> Comparações: {q_comp} | Movimentações: {q_mov}")

print("\n\n=== DESAFIO ADICIONAL (N = 1.000) ===")
n_desafio = 1000
vetor_base = [random.randint(1, 10000) for _ in range(n_desafio)]

cenarios = {
    "Aleatório": vetor_base,
    "Já Ordenado": sorted(vetor_base),
    "Inversamente Ordenado": sorted(vetor_base, reverse=True)
}

for nome, v_teste in cenarios.items():
    b_comp, b_trocas = bubble_sort(v_teste)
    i_comp, i_mov = insertion_sort(v_teste)
    s_comp, s_trocas = selection_sort(v_teste)
    q_comp, q_mov = quick_sort(v_teste)

    print(f"\n--- Cenário: {nome} ---")
    print(f"Bubble Sort -> Comparações: {b_comp} | Trocas: {b_trocas}")
    print(f"Insertion Sort -> Comparações: {i_comp} | Movimentações: {i_mov}")
    print(f"Selection Sort -> Comparações: {s_comp} | Trocas: {s_trocas}")
    print(f"Quick Sort -> Comparações: {q_comp} | Movimentações: {q_mov}")

```

Etapa 3 – Resultados

Tamanho	Bubble Comparações	Bubble e Trocas	Insertion Comparações	Insertion Mov.	Selection Comparações	Selection Trocas	Quick Comparações	Quick Mov.
10	45	19	27	28	45	7	29	19
20	190	84	99	103	190	16	58	43
1.000	499.500	239.681	240.670	240.680	499.500	992	10.385	5.850

Etapa 4 - Análise dos resultados

- a) O **Insertion Sort** teve o menor número de comparações para 10 elementos (27), seguido de perto pelo Quick Sort (29).
- b) O **Selection Sort** realizou menos trocas/movimentações, fazendo apenas 7 para o vetor de tamanho 10.
- c) Sim, a lógica seguiu a mesma proporção. No tamanho 20, o Selection continuou com pouquíssimas trocas (16) e o Quick mantendo a menor quantidade de comparações (58).
- d) Com 1.000 elementos, o volume de operações explodiu nos algoritmos simples (chegando a quase 500 mil comparações no Bubble e Selection), enquanto o Quick Sort se destacou fazendo apenas 10.385 comparações.
- e) Não. Apesar de todos apresentarem complexidade quadrática, o Selection faz um número fixo de comparações (499.500) e poucas trocas (992), enquanto o Insertion encerra os laços antecipadamente, totalizando 240.670 comparações.
- f) O **Bubble Sort** teve o maior crescimento geral, acumulando mais de 739 mil operações no total (comparações + trocas) para 1.000 dados.
- g) O Quick Sort cresceu de forma muito mais lenta que os outros. Enquanto os demais passaram de centenas de milhares de operações, ele estagnou na casa das 10 mil comparações.
- h) Sim. Os resultados mostram claramente a diferença na prática entre a complexidade quadrática dos três primeiros e a logarítmica do Quick Sort.
- i) Eu escolheria o **Quick Sort**. Como a central de distribuição processa milhares de pedidos, a diferença entre 10 mil e 500 mil operações impacta diretamente na velocidade do sistema.

Desafio Adicional

Resposta da Análise do Desafio Adicional

A organização inicial dos dados interfere na quantidade de operações realizadas por todos os algoritmos da mesma maneira?

Não. A ordem inicial afeta cada algoritmo de uma forma diferente:

Insertion Sort: É o mais sensível ao estado inicial. Quando o vetor já veio ordenado, ele atingiu seu melhor caso $O(n)$, fazendo só 999 comparações.

Selection Sort: O número de comparações foi exatamente o mesmo (499.500) nos três cenários, pois ele sempre percorre a lista inteira para achar o menor valor. Apenas as trocas foram para zero no vetor ordenado.

Bubble Sort: Na versão simples usada no código, ele realizou 499.500 comparações independente da ordem inicial, zerando apenas as trocas quando já estava ordenado.

Quick Sort: Como o pivô escolhido foi fixo no último elemento, em vetores já ordenados ou em ordem inversa ele perdeu seu desempenho comum e degradou para o pior caso $O(n^2)$ (499.500 comparações).